
This document should help you explain the entire

drawModel.js

created by Berat Burak KAYA on 27.12.2024

Scene Setup

1. **Creating the Scene and Camera**

```
const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(
  75,
  window.innerWidth / window.innerHeight,
  0.1,
  1000
);
```

   - **Scene**: The

THREE.Scene

object is the container for all the objects, lights, and cameras in your 3D world.

- **Camera**: The

THREE.PerspectiveCamera

is used to simulate the perspective of a human eye. The parameters are:
- Field of view (75 degrees)
- Aspect ratio (window width divided by window height)
- Near clipping plane (0.1)
- Far clipping plane (1000)

2. **Initializing the Renderer**

```
const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(window.innerWidth, window.innerHeight);
renderer.setPixelRatio(window.devicePixelRatio > 1 ? 2 : 1);
```

   - **Renderer**: The

THREE.WebGLRenderer

is responsible for rendering the scene to the canvas element. The `antialias` option is enabled for smoother edges.

- **setSize**: Sets the size of the rendering window.
- **setPixelRatio**: Adjusts the pixel ratio for high-DPI screens.

3. **Tone Mapping and Background Color**

```
renderer.toneMapping = THREE.ReinhardToneMapping;
renderer.toneMappingExposure = 2.0;
renderer.setClearColor(0x95b7b7);
```

   - **Tone Mapping**: Reinhard tone mapping is used to map high dynamic range (HDR) values to a displayable range, balancing bright and dark parts of the scene.
   - **Exposure**: Adjusts the exposure level (default is 1.0).
   - **Background Color**: Sets the background color of the scene to a light blue (hex color `0x95b7b7`).

4. **Appending Renderer to Document**

```
document.body.appendChild(renderer.domElement);
```

   - Appends the renderer's canvas element to the HTML document's body.

Lighting

5. **Directional Light**

```
const directionalLight = new THREE.DirectionalLight(0xffffff, 1);
directionalLight.position.set(5, 10, 7).normalize();
scene.add(directionalLight);
```

   - **Directional Light**: Simulates sunlight with a white color (`0xffffff`) and intensity of 1.
   - **Position**: Sets the light's position above and to the side of the scene.
   - **Normalize**: Normalizes the light's position vector.

6. **Ambient Light**

```
const ambientLight = new THREE.AmbientLight(0x404040, 1.5);
scene.add(ambientLight);
```

   - **Ambient Light**: Provides a soft light that affects all objects equally, ensuring no part of the scene is too dark. The color is a soft gray (`0x404040`) with an intensity of 1.5.

Additional Code

7. **Commented Sphere Geometry**

```
// const geometry = new THREE.SphereGeometry(2, 32, 32);
// const material = new THREE.MeshStandardMaterial({
//   color: 0x888888,
//   transparent: true,
//   opacity: 0.5,
//   metalness: 0.7,
//   roughness: 0.2,
// });
// const sphere = new THREE.Mesh(geometry, material);
// scene.add(sphere);
```

- This section is commented out, but if uncommented, it would create a sphere geometry with a radius of 2 and 32 segments along both the width and height, and add it to the scene.

Model Loading

8. **Loading a GLB Model**

```
const loader = new THREE.GLTFLoader();

// Create a group to hold both the model and ground, gridHelper
const group = new THREE.Group();

loader.load(
  "/models/truck.glb",
  function (gltf) {
    const model = gltf.scene;

    model.position.set(0, 0.5, 0); // Adjust position over grid to avoid clipping
    model.scale.set(2, 2, 2); // Adjust scale if needed

    // Apply high-quality materials
    model.traverse(function (node) {
      if (node.isMesh) {
        node.material = new THREE.MeshStandardMaterial({
          color: 0x999999,
          alphaTest: 0.5,
          transparent: true,
          side: THREE.DoubleSide,
          opacity: 0.5,
          wireframeLinewidth: 50,
          metalness: 0.5,
          roughness: 0.1,
        });

        console.log("material name : ", node);

        // Highlight the mesh by changing its material color
        node.material.color.set(Math.random() * 0xffffff); // Random color for each part
      }
    });

    // Calculate the bounding box of the model
    const box = new THREE.Box3().setFromObject(model);

    // Position arrows at the ends of the bounding box
    const _position = new THREE.Vector3(box.min.x, box.min.y, box.min.z);

    calculateDimensions(box);

    group.add(model);

    scene.add(group);

    animate();
  },
  undefined,
  function (error) {
    console.error("Error loading model:", error);
  }
);
```

- **GLTFLoader**: Loads a GLB model from the specified path.
- **Group**: Creates a group to hold the model and other objects.
- **load**: Loads the model and applies transformations and materials.
- **Bounding Box**: Calculates the bounding box of the model.
- **calculateDimensions**: Calls the function to calculate and display the dimensions of the model.
- **Add to Scene**: Adds the model to the group and the group to the scene.

Text Label Creation

9. **Function to Create a Text Label**

```
function createTextLabel(text, position, color, mirror, rotate) {
  const loader = new THREE.FontLoader();
  loader.load(
    "https://threejs.org/examples/fonts/helvetiker_regular.typeface.json",
    function (font) {
      const textGeo = new THREE.TextGeometry(text, {
        font: font,
        size: 0.1,
        height: 0.01,
      });
      const textMaterial = new THREE.MeshBasicMaterial({ color: color });
      const textMesh = new THREE.Mesh(textGeo, textMaterial);
      textMesh.scale.set(
        mirror[0] ? -1 : 1,
        mirror[1] ? -1 : 1,
        mirror[2] ? -1 : 1
      );
      textMesh.position.copy(position);
      textMesh.rotation.set(rotate[0], rotate[1], rotate[2]);
      scene.add(textMesh);
    },
    undefined,
    function (error) {
      console.error("Error loading font:", error);
    }
  );
}
```

- **createTextLabel**: This function creates a 3D text label in the scene.
  - **Parameters:**

text

: The text to display.
-

position

: The position of the text in the scene.
-

color

: The color of the text.
-

mirror

: An array indicating whether to mirror the text on the X, Y, and Z axes.
-

rotate

: An array indicating the rotation of the text on the X, Y, and Z axes.
- **FontLoader**: Loads the font from a URL.
- **TextGeometry**: Creates the geometry for the text.
- **MeshBasicMaterial**: Creates a basic material with the specified color.
- **Mesh**: Combines the geometry and material into a mesh.
- **Scale**: Mirrors the text if specified.
- **Position**: Sets the position of the text.
- **Rotation**: Sets the rotation of the text.
- **Add to Scene**: Adds the text mesh to the scene.

Line Creation

10. **Function to Create a Line**

```
function createLine(start, end, color) {
  const points = [start, end];
  const geometry = new THREE.BufferGeometry().setFromPoints(points);
  const material = new THREE.LineBasicMaterial({ color: color });
  const line = new THREE.Line(geometry, material);
  scene.add(line);
  return line;
}
```

- **createLine**: This function creates a line between two points in the scene.
  - **Parameters**:
    - `start`: The starting point of the line.
    - `end`: The ending point of the line.
    -

color

: The color of the line.
- **Points**: An array containing the start and end points.
- **BufferGeometry**: Creates a geometry from the points.
- **LineBasicMaterial**: Creates a basic material with the specified color.
- **Line**: Combines the geometry and material into a line.
- **Add to Scene**: Adds the line to the scene.
- **Return Line**: Returns the created line object.

Dimension Calculation

11. **Function to Calculate Dimensions**

```
function calculateDimensions(box) {
  const width = box.max.x - box.min.x;
  const height = box.max.y - box.min.y;
  const depth = box.max.z - box.min.z;
```

- **calculateDimensions**: This function calculates the dimensions (width, height, depth) of a bounding box.

Parameters:

box

: The bounding box object.
- **Width**: Calculates the width of the box.
- **Height**: Calculates the height of the box.
- **Depth**: Calculates the depth of the box.

12. **Creating Dimension Lines and Labels**

```
const lineX = createLine(
  new THREE.Vector3(box.min.x, box.min.y, box.min.z),
  new THREE.Vector3(box.max.x, box.min.y, box.min.z),
  0xff0000
); // Red for width

createTextLabel(
  `2.44 m`,
  new THREE.Vector3(box.max.x, box.min.y, box.min.z)
    .clone()
    .add(new THREE.Vector3(-box.max.x * 0.5, 0.05, 0)),
  0xff0000,
  [false, false, false],
  [0, Math.PI, 0]
);
```

- **Width Line and Label**: Creates a red line and label for the width dimension.
  - **createLine**: Creates a red line from the minimum to the maximum x-coordinate of the box.
  - **createTextLabel**: Creates a text label for the width dimension.

```
const lineY = createLine(
  new THREE.Vector3(box.min.x, box.min.y, box.min.z),
  new THREE.Vector3(box.min.x, box.max.y, box.min.z),
  0x00ff00
); // Green for height

createTextLabel(
  `${((height * 2.44 * 10) / 7.3).toFixed(2)} m`,
  new THREE.Vector3(box.min.x, box.getCenter().y, box.min.z)
    .clone()
    .add(new THREE.Vector3(0, 0, 0.05)),
  0x00ff00,
  [false, false, false],
  [0, -Math.PI / 2, 0]
);
```

- **Height Line and Label**: Creates a green line and label for the height dimension.
  - **createLine**: Creates a green line from the minimum to the maximum y-coordinate of the box.
  - **createTextLabel**: Creates a text label for the height dimension.

```
const lineZ = createLine(
  new THREE.Vector3(box.min.x, box.min.y, box.min.z),
  new THREE.Vector3(box.min.x, box.min.y, box.max.z),
  0x0000ff
); // Blue for depth

createTextLabel(
  `${((depth * 2.44 * 10) / 7.3).toFixed(2)} m`,
  new THREE.Vector3(box.min.x, box.min.y, box.getCenter().z)
    .clone()
    .add(new THREE.Vector3(0, 0.05, 0)),
  0x0000ff,
  [true, false, true],
  [0, Math.PI / 2, 0]
);
```

- **Depth Line and Label**: Creates a blue line and label for the depth dimension.
    - **createLine**: Creates a blue line from the minimum to the maximum z-coordinate of the box.
    - **createTextLabel**: Creates a text label for the depth dimension.

13. **Adding Lines to Scene**

```
scene.add(lineX);
scene.add(lineY);
scene.add(lineZ);
```

- **Add to Scene**: Adds the created lines to the scene.

Camera and Controls

14. **Camera Position**

```
camera.position.set(0, 2, 3);
camera.lookAt(scene.position);
```

- **Camera Position**: Sets the initial position of the camera and ensures it is looking at the center of the scene.

15. **Orbit Controls**

```
const controls = new THREE.OrbitControls(camera, renderer.domElement);
controls.rotateSpeed = 0.5;
controls.zoomSpeed = 0.5;
controls.panSpeed = 0.5;
controls.enableDamping = true;
controls.dampingFactor = 0.05;
```

- **OrbitControls**: Adds orbit controls to allow the user to rotate, zoom, and pan the camera.
    - **rotateSpeed**: Adjusts the speed of rotation.
    - **zoomSpeed**: Adjusts the speed of zooming.
    - **panSpeed**: Adjusts the speed of panning.
    - **enableDamping**: Enables damping (inertia) for smoother controls.
    - **dampingFactor**: Sets the damping factor.

Grid Helper

16. **Grid Helper**

```
const gridSize = 6;
const gridDivisions = 15;
const gridHelper = new THREE.GridHelper(
  gridSize,
  gridDivisions,
  0x808080,
  0x808080
);
group.add(gridHelper);
```

- **GridHelper**: Adds a grid helper to the scene to visualize the ground plane.
    - **gridSize**: Sets the size of the grid.
    - **gridDivisions**: Sets the number of divisions in the grid.
    - **gridHelper**: Creates the grid helper with specified size and divisions.

Ground Plane (Commented Out)

17. **Ground Plane (Commented Out)**

```
// const groundGeometry = new THREE.PlaneGeometry(6, 6);
// const groundMaterial = new THREE.MeshStandardMaterial();
// const ground = new THREE.Mesh(groundGeometry, groundMaterial);
// ground.rotation.x = -Math.PI / 2;
```

- **Ground Plane**: This section is commented out, but if uncommented, it would create a ground plane.
    - **PlaneGeometry**: Creates a plane geometry with specified width and height.
    - **MeshStandardMaterial**: Creates a standard material for the plane.
    - **Mesh**: Combines the geometry and material into a mesh.
    - **Rotation**: Rotates the plane to lie flat (facing up).

Animation Loop

18. **Animation Function**

```
function animate() {
  requestAnimationFrame(animate);
  renderer.render(scene, camera);
}
animate();
```

- **animate**: This function creates an animation loop.
    - **requestAnimationFrame**: Requests the browser to call the

animate

function before the next repaint.
- **render**: Renders the scene from the perspective of the camera.

Window Resize Handling

19. **Window Resize Event Listener**

```
window.addEventListener('resize', onWindowResize, false);

function onWindowResize() {
  camera.aspect = window.innerWidth / window.innerHeight;
  camera.updateProjectionMatrix();
  renderer.setSize(window.innerWidth, window.innerHeight);
}
```

- **resize**: Adds an event listener to handle window resize events.
  - **onWindowResize**: Adjusts the camera's aspect ratio and updates the renderer size when the window is resized.

Example Usage

20. **Example Usage of Functions**

```
const start = new THREE.Vector3(-1, 0, 0);
const end = new THREE.Vector3(1, 0, 0);
createLine(start, end, 0xff0000);

const textPosition = new THREE.Vector3(0, 1, 0);
createTextLabel('Hello, World!', textPosition, 0x00ff00, [false, false, false], [0, 0, 0]);
```

- **createLine**: Creates a red line from `(-1, 0, 0)` to `(1, 0, 0)`.
- **createTextLabel**: Creates a green text label "Hello, World!" at position `(0, 1, 0)` without mirroring or rotation.